

Lab 02 — Praktijklab: een image ontleden, een registry opzetten

Doel: werken met tags, lagen en digests, daarna een image publiceren in een private registry die je zelf draait — en onderweg ontdekken waarom Podman je namen voluit laat schrijven.

Vereisten — Lab 01 afgewerkt (rootless Podman onder WSL, `systemd` actief). Poort `5000` moet vrij zijn (`ss -lntp | grep :5000` mag niets teruggeven).

Geleverde bestanden — `files/Dockerfile` (twee regels, uitgelegd in lab 04; hier dient het alleen om images mee te genereren).

Stap 1 — Een imagenaam lezen

```
podman pull nginx:alpine
podman pull alpine
```

Observeer `Resolved "nginx" as an alias`, dan `Trying to pull docker.io/library/nginx:alpine...` en de regels `Copying blob`. Voor `alpine` verschijnt alleen een kale ID: die image staat er al sinds lab 01.

```
podman image inspect --format '{{.RepoTags}}' nginx:alpine
podman image inspect --format '{{.Digest}}' nginx:alpine
```

Observeer het contrast: het eerste commando geeft `[docker.io/library/nginx:alpine]` — de **volledige** naam, die je nooit getypt hebt — het tweede geeft `sha256:1f25fedd50aec27413031afb...`.

Uitleg. `nginx:alpine` is een leesbare, verplaatsbare naam; de digest is de echte, permanente identiteit van de inhoud. Podman toont altijd de volledige naam: in zijn opslag bestaat er geen "korte naam", alleen in jouw commando.

Probeer nu een naam die niet in de aliaslijst staat:

```
grep -c '=' /etc/containers/registries.conf.d/000-shortnames.conf
grep -E '^s*(alpine|nginx|eclipse-temurin)' /etc/containers/registries.conf.d/000-
shortnames.conf
grep unqualified-search-registries /etc/containers/registries.conf
```

Observeer dat `alpine` en `nginx` een alias hebben, `eclipse-temurin` niet, en dat de zoeklijst van Ubuntu alleen `docker.io` bevat — daarom werkt `podman pull eclipse-temurin:21-jre-alpine` bij jou toch, terwijl je op Fedora een vraag zou krijgen.

PODMAN

Korte namen zijn handig in de terminal, maar geen praktijk voor productie. Schrijf in een Dockerfile of een script `docker.io/library/eclipse-temurin:21-jre-alpine`: dan hangt het resultaat niet meer af van de configuratie van de machine die het uitvoert.

Stap 2 — Je images olijsten

```
podman images
```

Observeer de kolommen `REPOSITORY / TAG / IMAGE ID / CREATED / SIZE`, met repositories die voluit geschreven zijn: `docker.io/library/nginx`, `docker.io/library/alpine`.

```
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'
podman images --format '{{.Repository}}:{{.Tag}}'
```

Uitleg. Maak van `--format` een gewoonte: je commando's hangen dan niet meer af van de weergave, en ze zijn bruikbaar in scripts.

Stap 3 — De lagen, en waar het gewicht naartoe gaat

```
podman history nginx:alpine --format 'table {{.Size}}\t{{.CreatedBy}}'
```

Observeer één grote laag (`50.7MB`, de installatie van nginx), enkele kleine `COPY`-lagen met scripts, een laag van `8.7MB` (Alpine) helemaal onderaan, en veel regels van `0B`.

Uitleg. De regels van `0B` zijn **metadata**-instructies: `ENV`, `CMD`, `EXPOSE`, `ENTRYPOINT`, `STOPSIGNAL`. Ze maken geen enkel bestand aan. Voer dit commando als eerste uit wanneer een image onverwacht zwaar is: de boosdoener springt er meteen uit.

Podman kan de lagen ook als een boom tonen, samen met hun bronimage:

```
podman image tree nginx:alpine
```

Observeer de eerste laag, gemarkeerd `Top Layer of: [docker.io/library/alpine:latest]`: nginx:alpine is **gebouwd op** de alpine-image die je al hebt — die laag van 8,7 MB staat maar één keer op schijf.

```
podman system df
podman system df -v | head -n 12
```

Observeer in de uitgebreide modus de kolom `SHARED SIZE`: `8.698MB` voor `nginx` én voor `alpine` — dezelfde laag, in beide meegeteld.

Stap 4 — `podman tag` kopieert niets

```
podman tag nginx:alpine mijn-nginx:v1
podman tag nginx:alpine mijn-nginx:preprod
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}' | grep nginx
```

Observeer drie regels... allemaal met **dezelfde** `IMAGE ID` — en je twee nieuwe namen met het voorvoegsel `localhost/`.

```
podman rmi mijn-nginx:v1
```

Observeer de uitvoer: alleen `Untagged: localhost/mijn-nginx:v1`. Geen `Deleted:`.

```
podman rmi mijn-nginx:preprod
```

Observeer opnieuw alleen `Untagged:` — want `docker.io/library/nginx:alpine` verwijst nog altijd naar de image.

Uitleg. Een tag is een verwijzing. Zolang er één naam overblijft, blijft de data staan. Daarom hoor je zo vaak "ik heb `rmi` gedaan en kreeg geen schijfruimte terug" — dat is volkomen normaal gedrag. En dat `localhost/`: een image die je zonder registry benoemt, hoort bij geen enkele registry, en Podman zegt dat er ook bij.

Stap 5 — Twee versies van dezelfde image bouwen

Kopieer het geleverde bestand naar een werkmap:

```
mkdir -p ~/labo-docker/02 && cd ~/labo-docker/02
cp <pad-van-het-lab>/files/Dockerfile .
cat Dockerfile
```

```
podman build -t demo-lagen:1.0 .
podman history demo-lagen:1.0 --format 'table {{.ID}}\t{{.Size}}\t{{.CreatedBy}}'
```

Observeer de regels `STEP 1/2`, `STEP 2/2`, `COMMIT demo-lagen:1.0` en `Successfully tagged localhost/demo-lagen:1.0`, en dan drie lagen: je `RUN` (2.05kB), de `CMD` van de basisimage (0B), en het Alpine-bestandssysteem (8.7MB), gemarkeerd `<missing>` omdat die laag bij een andere image hoort.

Wijzig de versie en bouw opnieuw op **dezelfde tag**:

```
sed -i 's/version 1/version 2/' Dockerfile
podman build -t demo-lagen:1.0 .
podman run --rm demo-lagen:1.0 cat /version.txt
```

Observeer `version 2`, en een nieuwe `IMAGE ID` achter dezelfde tag.

```
podman images --filter dangling=true
```

Observeer een regel `<none> <none>` met de **oude** `IMAGE ID`: dat is de *dangling* image, degene die haar naam kwijtraakte. Ze neemt nog altijd 8,7 MB in (gedeeld, in dit geval).

Uitleg. De tag `demo-lagen:1.0` is **verplaatst** naar een nieuwe image — precies het scenario uit vraag 2, en niets heeft je gewaarschuwd. Ruim het restant op via zijn ID:

```
podman rmi $(podman images --filter dangling=true -q)
```

Stap 6 — Een private registry opzetten

Een registry is gewoon een container:

```
podman run -d -p 5000:5000 --name lab-registry registry:2
podman ps --filter name=lab-registry
curl -s http://localhost:5000/v2/_catalog
```

Observeer `0.0.0.0:5000->5000/tcp` in de kolom `PORTS`, en dan `{"repositories":[]}`: de registry is leeg en werkt.

→ WINDOWS / WSL

Poort 5000 is gepubliceerd binnen de WSL-VM, maar Windows ziet ze ook: open `http://localhost:5000/v2/_catalog` in je Windows-browser. WSL 2 stuurt poorten waarop Linux luistert automatisch door naar `localhost` aan Windows-zijde (*localhost forwarding*). Zo kun je in de volgende labs de Angular-frontend testen vanuit Edge of Chrome.

Publiceer er je image in:

```
podman tag demo-lagen:1.0 localhost:5000/basis/demo:1.0
podman push localhost:5000/basis/demo:1.0
```

Observeer de mislukking:

```
Error: ... pinging container registry localhost:5000: Get "https://localhost:5000/v2/":
http: server gave HTTP response to HTTPS client
```

Uitleg. Je registry spreekt HTTP, en Podman eist standaard HTTPS — **zelfs voor localhost**, waar Docker stilzwijgend een uitzondering maakt. Bij een testregistry zeg je dat expliciet:

```
podman push --tls-verify=false localhost:5000/basis/demo:1.0
curl -s http://localhost:5000/v2/_catalog
curl -s http://localhost:5000/v2/basis/demo/tags/list
```

Observeer de regels `Copying blob` en `Writing manifest to image destination`, en dan `{"repositories":["basis/demo"]}` en `{"name":"basis/demo","tags":["1.0"]}`.

→ BEVEILIGING

Het alternatief is de registry declareren in `~/.config/containers/registries.conf` (`[[registry]]`, `location = "localhost:5000"`, `insecure = true`). Dat is handig op een ontwikkelmachine en gevaarlijk overal elders: "insecure" betekent dat noch de identiteit noch de versleuteling van de registry geverifieerd wordt, zodat een aanvaller op het netwerk een andere image kan onderschuiven. In een bedrijf heeft een registry een certificaat — punt uit.

In drie commando's heb je nagebootst wat de CI van je bedrijf doet. Controleer nu de differentiële overdracht:

```
podman tag demo-lagen:1.0 localhost:5000/basis/demo:1.1
podman push --tls-verify=false localhost:5000/basis/demo:1.1
```

Observeer dat dezelfde blobs vermeld worden, maar dat de overdracht meteen klaar is: de registry heeft ze al, dus alleen het manifest wordt geschreven.

Stap 7 — Pullen op digest

Haal de digest op zoals de registry hem kent:

```
curl -sI -H "Accept: application/vnd.oci.image.manifest.v1+json" \
  http://localhost:5000/v2/basis/demo/manifests/1.0 | grep -i docker-content-digest
```

Observeer een regel `Docker-Content-Digest: sha256:239accdd...`. Kopieer die waarde.

```
podman rmi localhost:5000/basis/demo:1.0
podman pull --tls-verify=false localhost:5000/basis/demo@sha256:<plak_hier>
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}\t{{.Digest}}' | grep demo
```

Observeer dat de image binnenkomt en dat `podman image inspect --format '{{.Digest}}' demo-lagen:1.0` precies de waarde geeft die je zonet plakte.

Uitleg. Serieuze uitrollen gebruiken deze vorm omdat ze **onvervalsbaar** is. Zelfs als iemand `basis/demo:1.0` opnieuw publiceert met een andere inhoud, blijft jouw digest wijzen naar de image die je getest hebt.

Stap 8 — Een image vervoeren zonder netwerk

```
podman save -o /tmp/demo.tar demo-lagen:1.0
ls -lh /tmp/demo.tar
tar -tf /tmp/demo.tar | head -n 4
```

Observeer een archief van `8.4M` met `<sha256>.tar`-bestanden (de lagen), een `.json`-bestand (de configuratie) en een `manifest.json`: het historische formaat `docker-archive`.

```
podman save --format oci-archive -o /tmp/demo-oci.tar demo-lagen:1.0
tar -tf /tmp/demo-oci.tar | head -n 4
```

Observeer deze keer `blobs/sha256/...` en `index.json`: de **OCI**-indeling, die elke tool (Docker, Podman, skopeo, Kubernetes) kan lezen.

Vergelijk met `export`, dat op een **container** werkt:

```
podman run -d --name tmpx nginx:alpine
podman export tmpx | podman import - nginx-plat:v1
podman image inspect --format '{{json .Config.Cmd}}' nginx-plat:v1
podman run --rm nginx-plat:v1
```

Observeer `null`, en dan de fout `crun: cannot find `` in $PATH`: de geïmporteerde image **weet niet meer wat ze moet starten**.

Uitleg. Onthoud de regel: `save` / `load` voor een image; `export` / `import` nooit voor een uitrol.

```
podman rmi demo-lagen:1.0 localhost:5000/basis/demo:1.1
podman load -i /tmp/demo.tar
```

Observeer `Loaded image: localhost/demo-lagen:1.0`: de tag reisde mee met het archief.

Opruimen

```
podman rm -f -t 0 tmpx lab-registry
podman rmi nginx-plat:v1 demo-lagen:1.0 registry:2
podman images --format '{{.Repository}}:{{.Tag}}' | grep -E 'demo|plat|registry'
rm -f /tmp/demo.tar /tmp/demo-oci.tar
```

De image die je in stap 7 op digest gepulld hebt, kan nog aanwezig zijn:

```
podman images --format 'table {{.ID}}\t{{.Repository}}' | grep localhost:5000
podman rmi <ID>
```

Observeer dat alleen `docker.io/library/alpine` en `docker.io/library/nginx:alpine` overblijven — die hou je bij voor de volgende labs.

Wat je nu moet kunnen beweren

- Een tag is een verplaatsbare verwijzing; de digest identificeert een inhoud. Podman bewaart en toont volledige namen, en zet `localhost/` voor die van jou.
- `podman history` en `podman image tree` onthullen waar het gewicht van een image naartoe gaat en wat ze deelt.
- `podman tag` dupliceert niets; `podman rmi` verwijdert eerst een naam, pas daarna data.
- Een `push` verstuurt alleen de lagen die in de registry ontbreken.
- Een registry is een gewone HTTP-dienst die je met één commando opzet — maar Podman eist TLS, tenzij je expliciet `--tls-verify=false` meegeeft.
- `export` / `import` vernietigt de configuratie van de image; `save` / `load` bewaart ze, in `docker-archive` - of `oci-archive` -formaat.